# LLM training, every type, three levels

Written 2026-09-08 for the Mercor screen and for general use. Each method gets: the plain version, how it actually works, the math, and where it shows up (Mercor's 397B recipe, your Catan bots). Sources are cited per section; the plain versions are mine.

Notation used throughout: x = prompt (or game state), y or o = the model's output (a sequence of tokens y_1… y_T), $\pi_\theta$ = the model being trained (the policy), $\pi_{ref}$ = a frozen copy used as an anchor, r = reward (a number), A = advantage (reward relative to a baseline), σ = sigmoid, E = average over data.

The one idea that organizes everything: **where does the training target come from?** If a human or teacher hands you the answer, it is supervised. If the model produces its own attempt and something scores it, it is reinforcement learning. Every method below is one of those two, or a hybrid that borrows a score to pick which examples to copy.

---

## 1. Pretraining (next-token prediction)

**Plain.** Show the model a mountain of text and make it guess the next word, over and over. Nothing else. It learns language, facts, and reasoning patterns as a side effect of getting good at guessing.

**How it works.** Every position in every document is a training example: the tokens so far are the input, the next token is the label. The loss is how surprised the model was by the real next token. Billions of documents, trillions of tokens, one objective.

**Math.** Cross-entropy over the sequence:

$$L(\theta) = -\Sigma_t \log \pi_\theta(x_t \mid x_{<t})$$

Minimizing this is the same as maximizing the likelihood of the data. The model outputs a probability distribution over the vocabulary at each step; the loss is the negative log of the probability it gave the correct token.

**Where.** Mercor's run starts from pretrained (and already post-trained) Qwen models; they do not pretrain. You will never pretrain either. Know it because everything downstream is a small nudge on top of this.

---

## 2. Supervised fine-tuning (SFT)

**Plain.** Same next-word game, but now on a small set of examples of the behavior you want: "here is a prompt, here is the answer a good assistant gives." Copy this.

**How it works.** Take demonstrations (prompt, response). Train with the same cross-entropy loss, but usually only on the response tokens, so the model learns to produce the answer rather than to predict the prompt. A few thousand to a few million examples. Same optimizer, tiny learning rate, one to a few epochs.

**Math.** Same loss, masked to the response:

$$L_{SFT}(\theta) = -\Sigma_{t \in \text{response}} \log \pi_\theta(y_t \mid x, y_{<t})$$

That is all. The teacher's tokens are the labels.

**Limits.** The model can only be as good as the demonstrations. It also learns the demonstrator's mistakes with equal enthusiasm. Off-distribution states (positions the teacher never faced) are not covered.

**Where.** Your Catan step 1: SFT an open model on the leaderboard winners' turns (view → action + reasoning). Mercor's "harness fixes alone gave +6 points" finding is a reminder that a lot of gain comes before any training, from formatting and prompting; SFT is the first training you'd add.

Source: any LLM fine-tuning tutorial; the loss is identical to pretraining (Ouyang et al. 2022, InstructGPT, arxiv.org/abs/2203.02155, Section 3.5 "Supervised fine-tuning").

---

## 3. Distillation

**Plain.** Use a big model as the teacher. Have it answer lots of prompts, then SFT a small model on those answers. The small model inherits the big one's behavior on those prompts.

**How it works.** Two flavors. Sequence-level (what you'd do): generate teacher outputs, treat them as SFT data. Logit-level: train the student to match the teacher's full probability distribution at each token, not just its chosen token; needs the teacher's logits, so needs an open teacher.

**Math.** Sequence-level distillation is exactly L_SFT on teacher outputs. Logit-level minimizes the KL between teacher and student distributions per position:

```
L_KD(θ) = Σ_t KL( p_teacher(· | x, y_<t) ‖ π_θ(· | x, y_<t) )
```

**Limits.** Same ceiling as SFT: the student approaches the teacher, rarely passes it. API providers' terms often restrict training competitors on their outputs; open-weight teachers avoid that.

**Where.** Your Catan plan is distillation of API models into an open model. Mercor's word for a related idea is "data": their thesis is that expert-built trajectories are the scarce input.

Source: Hinton et al. 2015 (arxiv.org/abs/1503.02531) for logit distillation; sequence-level distillation is Kim & Rush 2016 (arxiv.org/abs/1606.07947).

---

## 4. Rejection sampling / expert iteration / best-of-N

**Plain.** Generate lots of attempts, keep only the ones a scorer liked, SFT on the keepers. Copy, but only the good stuff. This is the cheapest way to get a score into a supervised recipe.

**How it works.** For each prompt, sample N outputs (from a teacher, or from the model itself). Score each (a verifier, a reward model, or the game outcome). Keep the top ones, or only correct ones. SFT on the kept set. Repeat: the improved model generates the next round. Repeating is called expert iteration or STaR-style self-improvement. Best-of-N at inference is the same trick without the training step.

**Math.** Let s(x, y) be the score. The kept set is $D^+ = \{(x, y) : s(x, y) \geq \tau\}$ or the top-k per prompt. Then apply L_SFT on $D^+$. Iterating: $\pi_{k+1}$ = argmin L_SFT over samples from $\pi_k$ filtered by s.

This is a crude policy improvement: you are approximating "increase probability of high-scoring outputs" without computing a gradient of the score.

**Limits.** Needs a scorer. Wastes the low-scoring samples (no signal from them). Can collapse diversity if you keep only the top sample each time.

**Where.** Your Catan step 2: keep only turns from games the teacher won. Mercor's post uses the same instinct in reverse: they filter out tasks that are broken (non-model errors) before training, so the learning signal is clean.

Source: Zelikman et al. 2022, STaR (arxiv.org/abs/2203.14465); Anthropic and Llama 2 both describe rejection-sampling fine-tuning (Llama 2 paper, arxiv.org/abs/2307.09288, Section 3.2.2).

---

## 5. Policy gradient basics (REINFORCE)

**Plain.** The model tries something. It gets a score. Make the tried thing more likely if the score was good, less likely if bad, in proportion to how good or bad. That is all RL for language models is; everything after this section is engineering to make it stable and cheap.

**How it works.** Sample an output y from π_θ. Compute reward r(x, y). Nudge the parameters so that log π_θ(y | x) goes up scaled by r. Because r can be any number, you subtract a baseline b (the average reward) so above-average outputs go up and below-average go down; otherwise every output goes up and nothing is learned.

**Math.** The objective is expected reward, J(θ) = E_{y ~ π_θ}[ r(x, y) ]. Its gradient (the log-derivative trick):

```
∇_θ J = E_{y ~ π_θ} [ ∇_θ log π_θ(y | x) · (r(x, y) − b) ]
```

The term (r − b) is the advantage A. For a sequence, log π_θ(y | x) = Σ_t log π_θ(y_t | x, y_<t), so the same advantage is applied to every token of the output unless you have per-token rewards.

**Limits.** High variance: one sample, one number, thousands of tokens. Unstable if updates are large. Hence PPO.

**Where.** This is the skeleton of GRPO. If you understand this equation you understand what Mercor's trainer is computing.

Source: Williams 1992 (REINFORCE); Sutton & Barto, Reinforcement Learning: An Introduction, Ch. 13.

---

## 6. PPO and RLHF (the 2022 recipe)

**Plain.** Policy gradient, but with two safety rails: don't move too far in one step (clipping), and don't drift too far from where you started (KL penalty). Plus a second model, the critic, that guesses how good each state is so the advantage is less noisy. RLHF is PPO where the score comes from a reward model trained on human preferences.

**How it works.** Three models in play: the policy π_θ, a frozen reference π_ref, and a value model V (the critic, usually the same size as the policy). Optionally a reward model. Loop: sample outputs; score them; compute advantages using V; update the policy with a clipped objective; update V to predict rewards better. The KL penalty keeps the policy near π_ref so it does not exploit the reward model or forget how to speak.

**Math.** Probability ratio per token between the new and old policy:

```
ρ_t = π_θ(y_t | x, y_<t) / π_old(y_t | x, y_<t)
```

Clipped surrogate objective:

```
L_PPO(θ) = E_t [ min( ρ_t A_t , clip(ρ_t, 1−ε, 1+ε) A_t ) ]
```

with ε ≈ 0.2. The min with the clipped term removes the incentive to push ρ beyond [1−ε, 1+ε]. RLHF adds the KL penalty into the reward:

```
r_total(x, y) = r_RM(x, y) − β · log [ π_θ(y | x) / π_ref(y | x) ]
```

Advantages A_t come from the critic via GAE (generalized advantage estimation), which blends TD errors δ_t = r_t + γ V(s_{t+1}) − V(s_t) over future steps.

**Reward model.** Trained on preference pairs (y_w preferred over y_l) with the Bradley–Terry loss:

```
L_RM(φ) = − E [ log σ( r_φ(x, y_w) − r_φ(x, y_l) ) ]
```

**Limits.** Four models in memory. The critic is expensive and hard to train for long text. Reward models get exploited (reward hacking): the policy finds outputs the RM likes that humans would not.

**Where.** GRPO exists because of the critic's cost. Mercor's post mentions PPO-family ideas only indirectly (clipping, ratios). Know the ratio and the clip; the rest of GRPO is a variation.

Source: Schulman et al. 2017, PPO (arxiv.org/abs/1707.06347); Ouyang et al. 2022, InstructGPT (arxiv.org/abs/2203.02155); Schulman 2020 on the KL estimator (joschu.net/blog/kl-approx.html).

---

## 7. DPO (direct preference optimization)

**Plain.** Skip the reward model and the RL loop. Take pairs "A is better than B," and directly push the model to prefer A over B relative to where it started. Preference learning with a plain supervised-looking loss.

**How it works.** DPO shows that the RLHF objective with a KL penalty has a closed-form optimal policy, and you can rewrite the reward in terms of the policy itself. Substituting into the Bradley–Terry loss gives a loss that only needs π_θ and π_ref on the two responses. No sampling during training, no critic.

**Math.**

```
L_DPO(θ) = − E_{(x, y_w, y_l)} [ log σ( β · ( log π_θ(y_w|x)/π_ref(y_w|x) − log π_θ(y_l|x)/
π_ref(y_l|x) ) ) ]
```

β controls how far from π_ref you are willing to go. The implicit reward is r(x, y) = β log π_θ(y|x)/π_ref(y|x).

**Limits.** Off-policy: the pairs come from somewhere else, so the model never sees its own mistakes. Can overfit to the pair distribution; tends to lower probability of both y_w and y_l in absolute terms while fixing their ordering.

**Where.** Your Catan branching idea: replay a transcript to a state, sample two continuations, roll both out, the winner's move is y_w. That gives DPO pairs from your own engine without an RL loop. Mercor did not use DPO in the 397B post.

Source: Rafailov et al. 2023 (arxiv.org/abs/2305.18290), Eq. 7.

---

## 8. GRPO (group relative policy optimization)

**Plain.** PPO without the critic. For one prompt, sample a group of answers, score them all, and grade each answer against its siblings: better than the group average goes up, worse goes down. The group replaces the critic as the baseline.

**How it works.** For each prompt x, sample G outputs {o_1…o_G} from the old policy. Score each: r_1…r_G. Normalize within the group to get advantages. Apply the PPO-style clipped ratio update with that advantage on every token of each output. Add a KL term to the reference directly in the loss (not in the reward). G is typically 8 to 64.

**Math.** Group-normalized advantage (outcome supervision):

```
A_i = ( r_i − mean(r_1…r_G) ) / std(r_1…r_G)
```

applied to every token t of output i. Objective:

```
L_GRPO(θ) = (1/G) Σ_i (1/|o_i|) Σ_t [ min( ρ_{i,t} A_i , clip(ρ_{i,t}, 1−ε, 1+ε) A_i ) − β
D_KL(π_θ ‖ π_ref) ]
```

with the per-token KL estimated by the unbiased, always-non-negative estimator:

```
D_KL ≈ π_ref(o_{i,t}|·)/π_θ(o_{i,t}|·) − log[ π_ref(o_{i,t}|·)/π_θ(o_{i,t}|·) ] − 1
```

**Why it works.** The mean over siblings is a good estimate of "how hard is this prompt," which is exactly what the critic was estimating. Dividing by std makes the scale of advantages the same across easy and hard prompts.

**Limits.** If all G outputs get the same reward, A_i = 0 for all of them: no gradient, wasted compute (DAPO fixes this). Long outputs and short outputs get different per-token weight depending on how you average (Section 10).

**Where.** Mercor's trainer is GRPO-family. Their DPPO vs GLM-5 loss ablation is about the ratio ρ under async training (Section 11).

Source: DeepSeekMath, Shao et al. 2024 (arxiv.org/abs/2402.03300), Section 4.1, Eq. 3 and the KL estimator right after it. Quotes checked 2026-09-08: "GRPO foregoes the need for additional value function approximation as in PPO, and instead uses the average reward of multiple sampled outputs, produced in response to the same question, as the baseline."

---

## 9. DAPO (four fixes on GRPO at scale)

**Plain.** ByteDance ran GRPO on a big model and hit four problems: the model's outputs became repetitive (entropy collapse), many prompts gave no signal, long answers were under-weighted, and truncated answers got punished unfairly. DAPO is one fix per problem.

**How it works and math.**

1. **Clip-Higher.** Use an asymmetric clip: [1 − ε_low, 1 + ε_high] with ε_high > ε_low (paper: 0.2 and 0.28). A symmetric clip caps how much a rare token's probability can grow, which starves exploration; raising the upper bound lets low-probability tokens rise.
2. **Dynamic Sampling.** Drop prompts where all G samples are correct or all wrong (accuracy 0 or 1), because A_i = 0 there. Keep sampling until the batch is full of prompts with mixed outcomes: 0 < Σ_i 1[correct_i] < G.
3. **Token-Level Policy Gradient Loss.** Average over all tokens in the batch instead of per sample:

   ```
   L = (1 / Σ_i |o_i|) Σ_i Σ_t [ … ]
   ```

   so long outputs contribute proportionally to their length. (Compare with GRPO's per-sample 1/|o_i|; see Section 10 for why Mercor went the other way.)
4. **Overlong Reward Shaping.** Instead of a hard zero for truncated outputs, a soft penalty that grows linearly with how far past the soft limit the output went, so the model is not punished for reasoning that was merely long.

Reward is rule-based: +1 if the final answer matches, −1 otherwise. Result: 50 on AIME 2024 with Qwen2.5-32B base.

**Where.** Mercor tried the DAPO-style ideas: overlong filtering and adaptive length penalty were "neutral or negative" for them, and token-level averaging lost to prompt-level. Different regime: math answers of similar length vs agent trajectories from 2k to 128k tokens. That contrast is a good thing to say out loud.

Source: DAPO, Yu et al. 2025 (arxiv.org/abs/2503.14476), Section 3.

---

## 10. Loss aggregation: token_mean vs prompt_mean

**Plain.** When a batch has a 2,000-token trajectory and a 128,000-token one, do they count equally, or does the long one count 64 times more? Token-mean says the long one dominates. Prompt-mean says each trajectory counts once.

**Math.** With per-token loss ℓ_{i,t}:

```
token_mean  = ( 1 / Σ_i |o_i| ) Σ_i Σ_t ℓ_{i,t}
prompt_mean = ( 1 / G ) Σ_i ( 1 / |o_i| ) Σ_t ℓ_{i,t}
```

**Where.** Mercor: prompt_mean beat token_mean by 3.9 points, because under token_mean the long trajectories dominated the gradient. DAPO argued the opposite for math. Same formula, opposite conclusions, different data.

Source: Mercor blog 2026-09-01, ablation section.

---

## 11. Asynchronous RL and off-policy corrections (DPPO, GLM-5 loss)

**Plain.** To keep the GPUs busy, the inference engines keep generating while the trainer updates weights, so some samples were produced by a slightly older policy (staleness), and the inference engine's numbers differ a bit from the trainer's (mismatch). Both variants correct for that using the probabilities the inference engine actually saw.

**How it works.** In fully-async training, a sample may be several updates old. The ratio ρ_t compares the current policy to whichever policy generated the sample; if you compute it against the inference engine's logprobs you correct both staleness and engine mismatch at once. Two ways to keep that ratio from blowing up: mask tokens where the two disagree too much, or truncate the ratio.

**Math (as Mercor describes them; no primary paper cited in the post).**

- Ratio against rollout logprobs: ρ_t = π_θ(y_t|·) / π_rollout(y_t|·), where π_rollout is what vLLM reported at sampling time.
- **DPPO**: mask token t (drop it from the loss) when |π_θ(y_t) − π_rollout(y_t)| exceeds a threshold, "via a binary approximation of total-variation divergence."
- **GLM-5 loss**: keep the token but clip the ratio: ρ_t ← min(ρ_t, c).

Mercor found the two within noise; DPPO pushed toward more, shorter turns. Their health check: mean |log π_trainer − log π_inference| below 0.03.

**Where.** This is where their bug lived (vLLM CPU offload + GDN + in-flight weight updates). If James is on the SkyRL work, this is the section he lives in.

Source: Mercor blog 2026-09-01; SkyRL (github.com/NovaSky-AI/SkyRL). If asked what DPPO stands for, say you only know it from their description.

---

## 12. RLVR (RL with verifiable rewards)

**Plain.** RL where the score is a hard check, not an opinion: did the answer match, did the tests pass, did the file end up right. No reward model to fool. The catch moves to the checker: if it can be shortcut, RL will find the shortcut.

**How it works.** Same GRPO or PPO loop; the reward function is a program or a rubric evaluated in the environment. Binary or per-criterion.

**Math.** r(x, y) = 1[verifier(x, y) passes], or a weighted sum of criteria Σ_k w_k c_k(x, y) ∈ [0, 1]. Then any policy-gradient method above.

**Where.** Mercor's whole product: expert-built tasks plus verifiers running in the sandbox. Their file-diff vs final-response finding is an RLVR lesson: grade the artifact, not the model's account of it. Your Catan step 3: win/lose is the verifier. It is sparse (one number per hundred decisions), so shaping (VP per turn) is tempting and gameable.

Source: Tülu 3, Lambert et al. 2024 (arxiv.org/abs/2411.15124), "Verifiable Rewards" section.

---

## 13. LoRA (how to fine-tune without touching every weight)

**Plain.** Instead of updating all the weights, freeze them and train a small add-on next to each big matrix. Cheap in memory, easy to swap. Edward Hu's invention; he leads model training at Mercor.

**How it works.** For a weight matrix W (d × k), add a low-rank product BA where B is d × r and A is r × k with r much smaller than d and k. Train only A and B. At inference merge them: W' = W + BA.

**Math.**

```
h = W x + (α / r) · B A x,    B ∈ R^{d×r}, A ∈ R^{r×k}, r ≪ min(d, k)
```

Trainable parameters drop from d·k to r·(d + k). A is initialized random, B to zero, so training starts from the original model.

**Where.** Your LIBERO-PRO result: LoRA underperformed full finetuning (15–21% vs 42%) at matched hyperparameters, which you read as the update needing to be spread across the network to undo trajectory memorization. That is a real, defensible data point about LoRA's limits, and Hu is on the team. Say it as a finding, not a verdict on LoRA.

Source: Hu et al. 2021 (arxiv.org/abs/2106.09685).

---

## 14. Self-play and on-policy vs off-policy (the Catan tie-in)

**Plain.** On-policy: the model learns from its own current attempts. Off-policy: it learns from someone else's, or its own older ones. SFT and DPO are off-policy; PPO and GRPO are on-policy. Self-play is on-policy RL where the opponents are copies of the model, so the environment gets harder as the model improves.

**How it works for Catan.** Start from the SFT'd model. Seat four copies. Play. Reward = win. GRPO over turns or over whole games. Periodically freeze a copy as the opponent pool so the model does not chase a moving target too fast. Log promises made and kept from the negotiation channel to score honesty separately from winning.

**Math.** Same as Sections 5 and 8 with r = game outcome. The credit-assignment problem: one terminal reward spread over T ≈ 100+ decisions, so A_t is the same for every token in the game unless you add a value estimate or a per-turn shaping term.

**Where.** Mercor's environment thesis in miniature. If asked, the honest version: "we're at step 1 (distill), step 3 (self-play RL) is the plan and the compute question."

Source: AlphaGo Zero, Silver et al. 2017, for self-play (nature.com/articles/nature24270); the rest is Sections 5 and 8.

---

## 15. Reward hacking, in one place

**Plain.** RL finds whatever the scorer rewards, including things you did not mean. Four patterns, one fix each.

| Pattern | Example | Fix |
|---|---|---|
| Grading narration, not state | Model says "done," file unchanged | Diff the artifact (Mercor's file-diff finding) |
| One terminal score | Model games the final number | Dense per-criterion rewards; your Merlyn VLM judges |
| Judge exploitation or drift | Policy learns what the LLM judge likes | Calibrate judge vs human labels on a held-out slice; track agreement |
| Train/eval overlap | Model memorizes the eval | Held-out worlds, not just held-out tasks |

**Where.** Every Mercor JD says "verifiers hard to game." Their eval-systems post names resisting gaming as constraint one.

---

## 16. Cheat sheet: one line each, for saying out loud

- Pretraining: predict the next token on everything; cross-entropy.
- SFT: same loss on demonstrations; copy the teacher.
- Distillation: SFT on a bigger model's outputs; can't pass the teacher.
- Rejection sampling: generate, score, keep the good ones, SFT; the cheap way to use a score.
- REINFORCE: push up what scored well, scaled by (reward − baseline).
- PPO: REINFORCE with a clipped ratio and a critic; RLHF adds a reward model and a KL leash.
- DPO: preference pairs, closed form, no reward model, no sampling.
- GRPO: PPO without the critic; siblings are the baseline; advantage = (r − mean)/std.
- DAPO: clip-higher, dynamic sampling, token-level loss, soft overlong penalty.
- token_mean vs prompt_mean: whether long trajectories dominate; Mercor: prompt_mean +3.9.
- Async corrections: ratio against rollout logprobs; DPPO masks, GLM-5 loss truncates; within noise.
- RLVR: reward from a checker; the risk moves to the checker.
- LoRA: freeze W, train BA with small r; your LIBERO-PRO result says it wasn't enough there.
- Self-play: on-policy RL against copies of yourself; Catan step 3.

**Self-check (from memory):** write the SFT loss; write the REINFORCE gradient; write the GRPO advantage; say what the critic does and why GRPO drops it; name DAPO's four; write DPO's loss in words; say why prompt_mean beat token_mean for Mercor; write LoRA's update.